



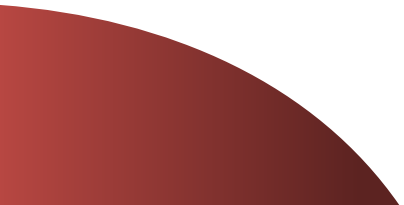
Unit-3

Memory Management



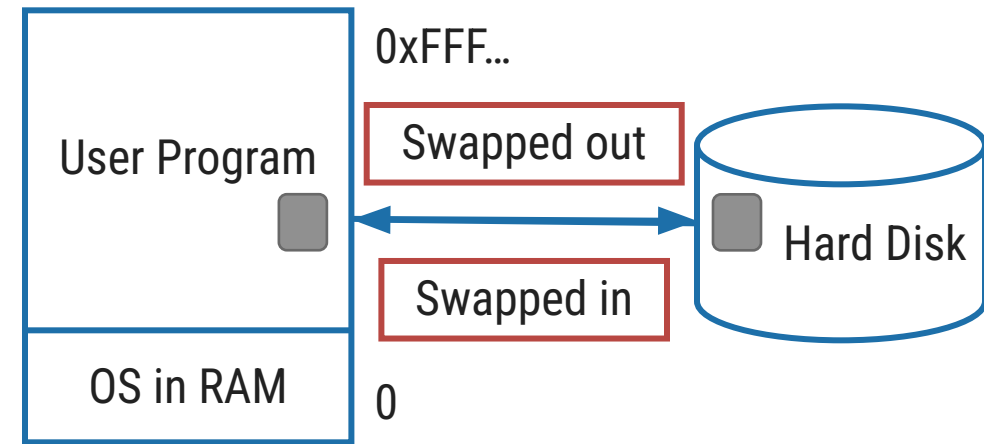


swapping



swapping

- What if we want to run multiple programs?
 1. OS saves entire memory on disk
 2. OS brings next program
 3. OS runs next program
- We can **use swapping** to **run multiple programs concurrently**.
- The **process of bringing in each process in its entirety in to memory, running it for a while and then putting it back on the disk** is called swapping.

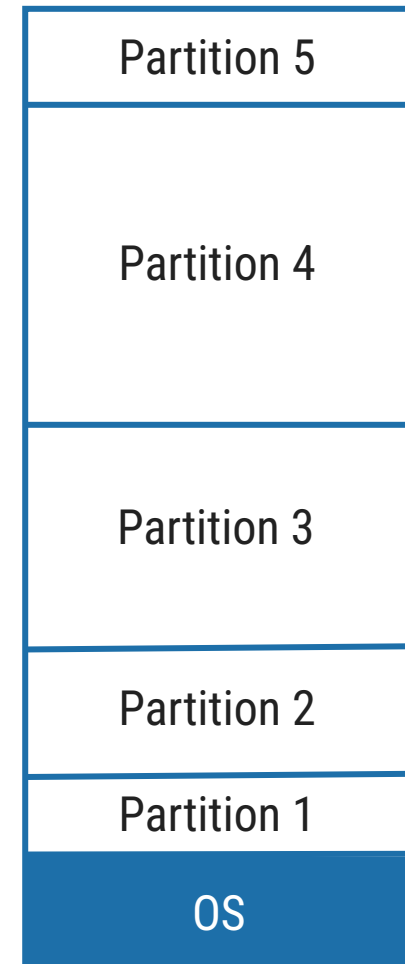


Ways to implement swapping system

- Two different ways to implement swapping system
 1. Multiprogramming with **fixed partitions**
 2. Multiprogramming with **dynamic partitions**

Multiprogramming with fixed partitions

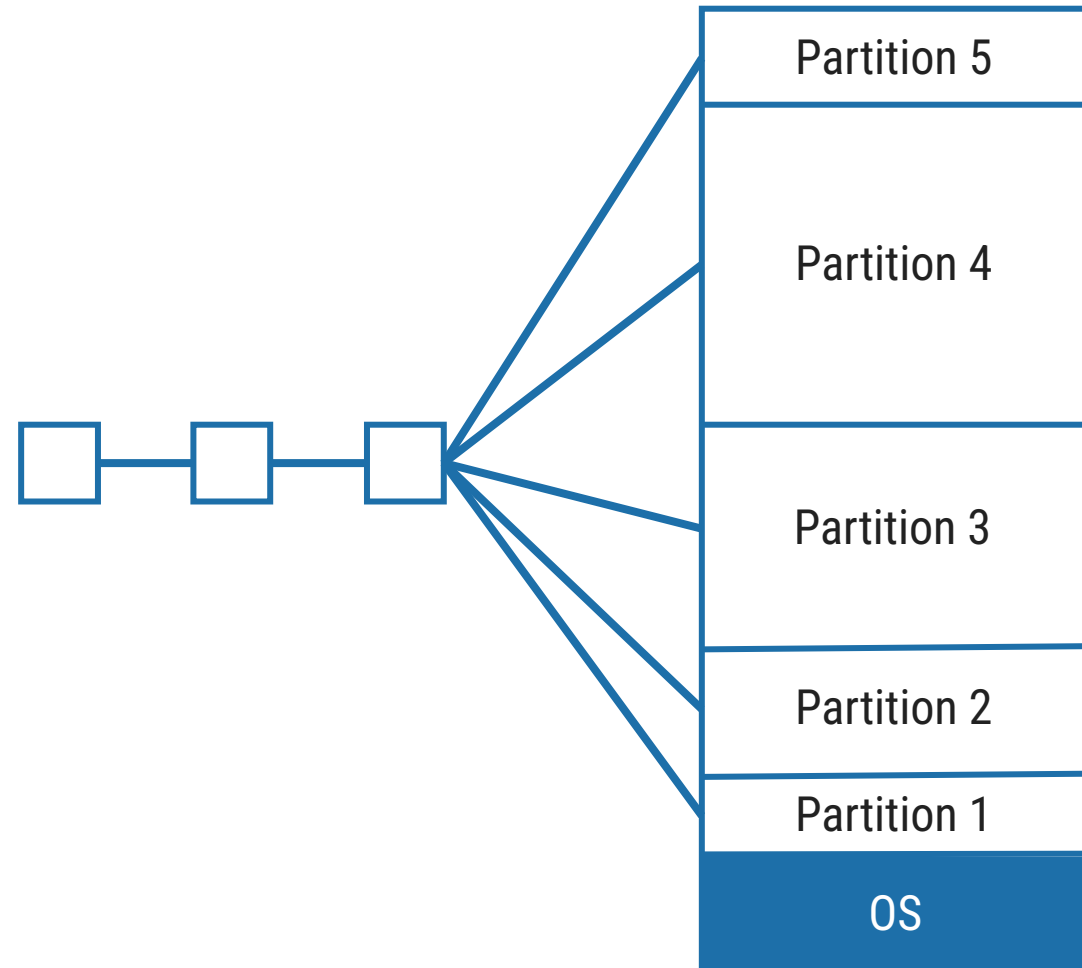
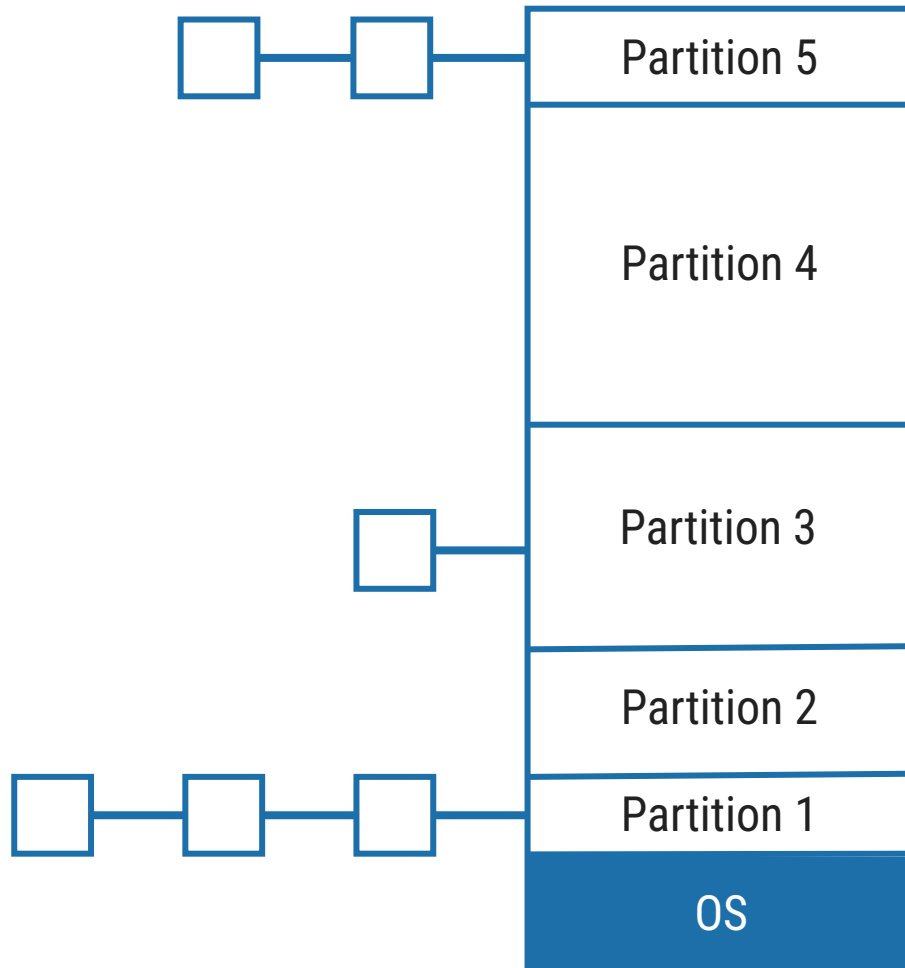
- ❑ Here memory is **divided into fixed sized partitions**.
- ❑ **Size can be equal or unequal** for different partitions.
- ❑ Generally **unequal partitions are used** for **better utilizations**.
- ❑ **Each partition can accommodate exactly one process**, means only single process can be placed in one partition.
- ❑ The partition **boundaries are not movable**.
- ❑ Whenever any **program needs to be loaded in memory, a free partition big enough to hold the program is found**. This **partition will be allocated** to that program or process.
- ❑ If **there is no free partition available of required size**, then the **process needs to wait**. Such **process will be put in a queue**.



Multiprogramming with fixed partitions

□ There are two ways to maintain queue

1. Using Multiple Input Queues



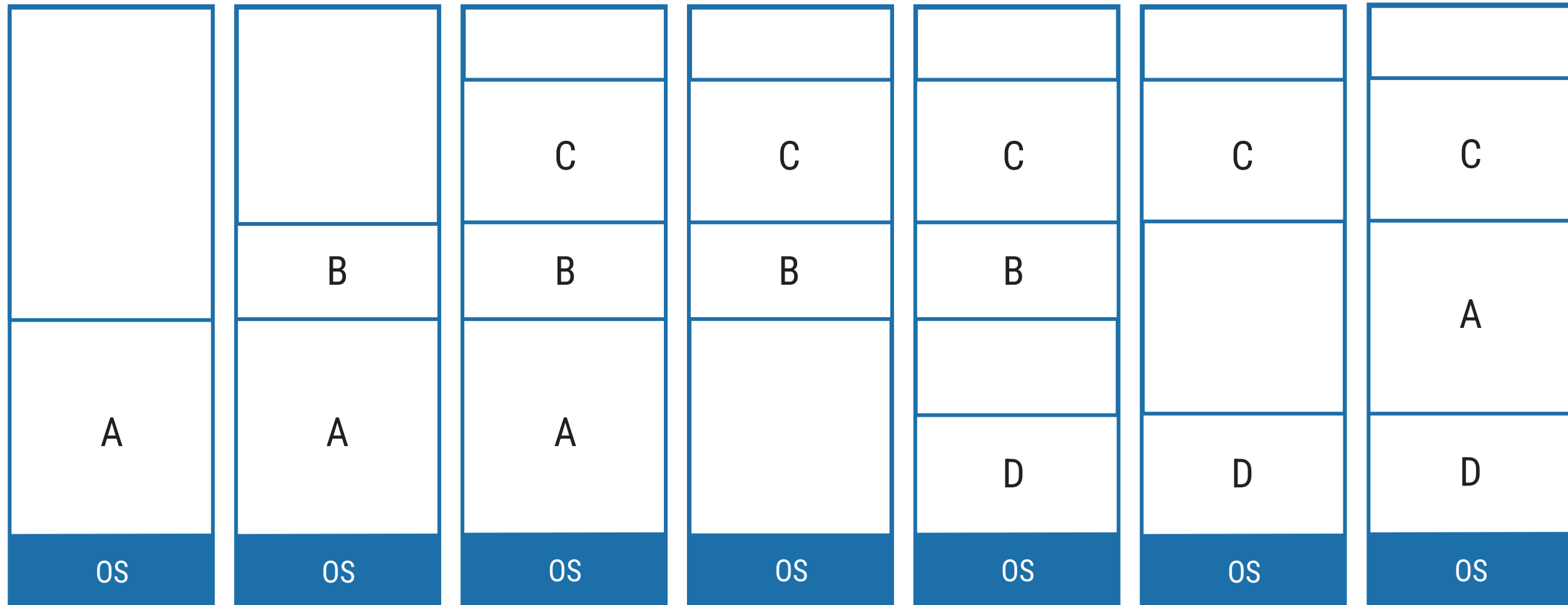
2. Using Single Input Queue

Multiprogramming with dynamic partitions

- ❑ Here, memory is **shared among operating system** and various simultaneously **running processes**.
- ❑ Initially, the **entire available memory is treated as a single free partition**.
- ❑ **Process is allocated exactly as much memory as it requires**.
- ❑ Whenever any **process enters** in a system, a **chunk of free memory big enough to fit the process is found and allocated**. The **remaining unoccupied space is treated as another free partition**.
- ❑ If **enough free memory is not available to fit the process, process needs to wait until required memory becomes available**.
- ❑ Whenever any **process gets terminate**, it **releases the space occupied**. If the **released free space is contiguous to another free partition, both the free partitions are merged together in to single free partition**.
- ❑ **Better utilization** of memory than fixed sized size partition.

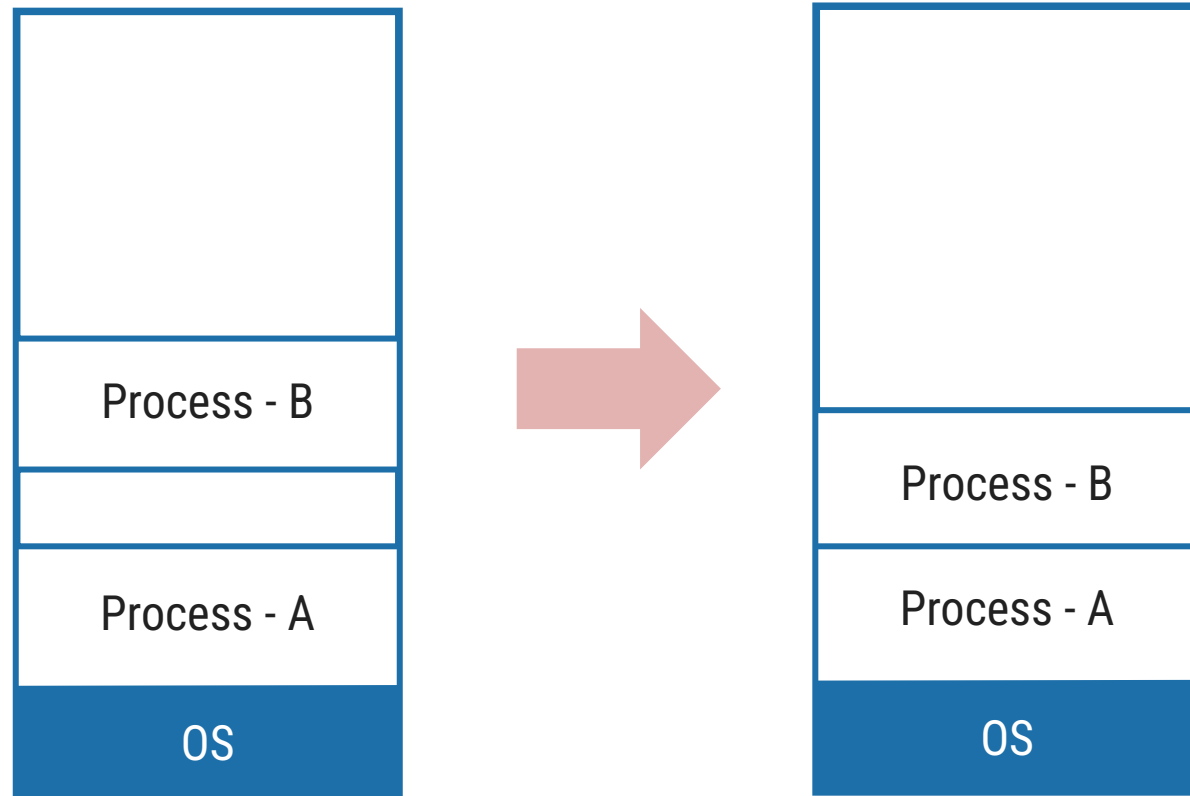


Multiprogramming with dynamic partitions



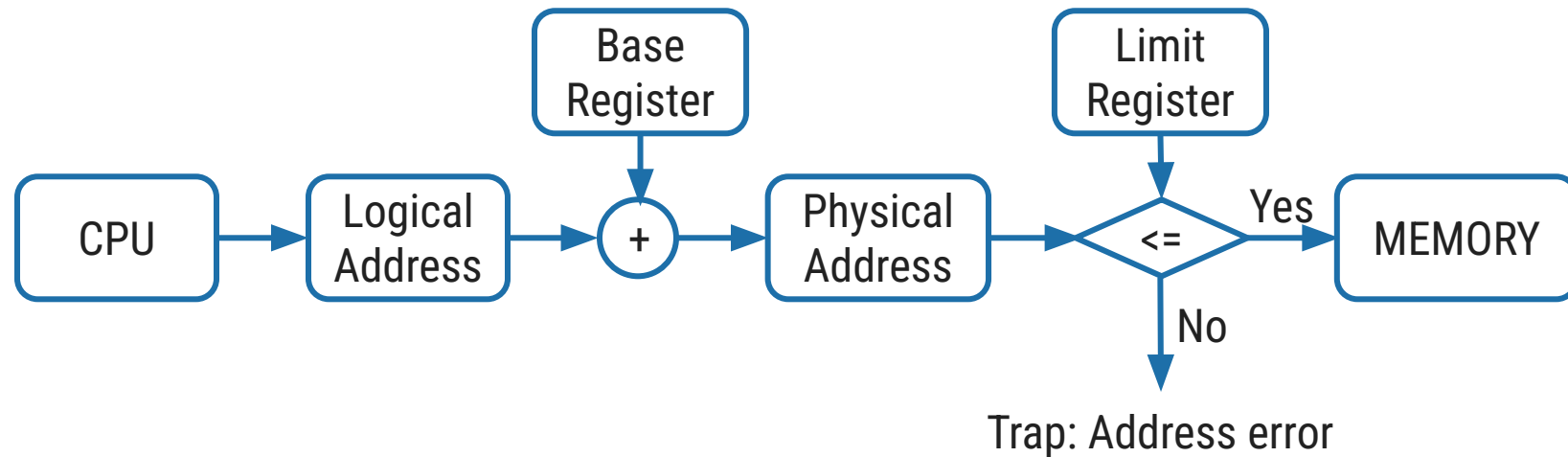
Memory compaction

- When **swapping creates multiple holes** in memory, it is possible to **combine them all in one big hole by moving all the processes downward** as far as possible. This technique is known as memory compaction.
- It **requires lot of CPU time**.



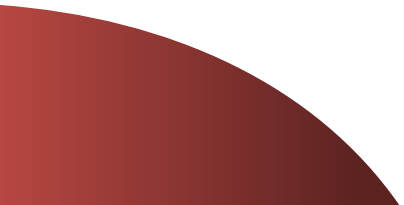
Dynamic relocation

- Dynamic relocation refers to **address transformations being done during execution of a program.**
- Steps in dynamic relocation
 1. Hardware **adds relocation register (base)** to virtual address to get a physical address
 2. Hardware **compares address with limit register**; address **must be less than or equal limit**
 3. If **test fails**, the **processor takes an address trap** and **ignores the physical address.**





Memory allocation



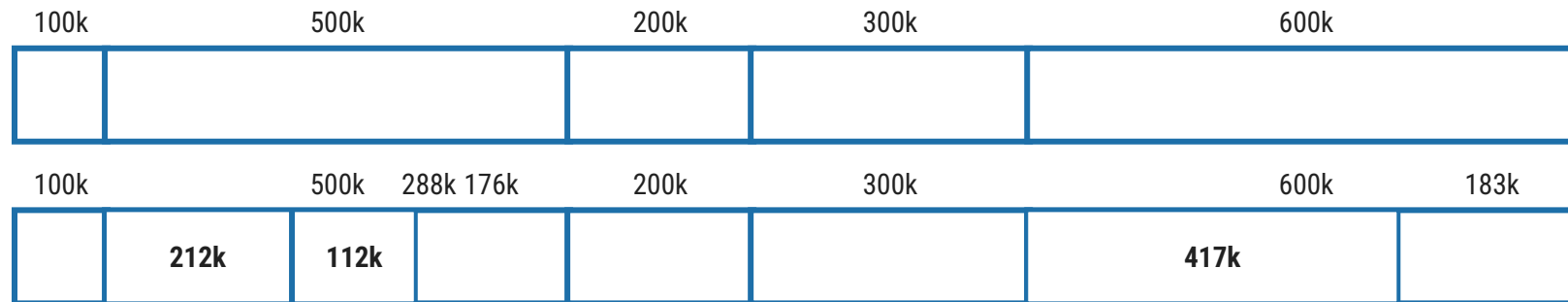
Memory allocation

□ Four memory allocation algorithms are as follow

1. First fit
2. Next fit
3. Best fit
4. Worst fit

First fit

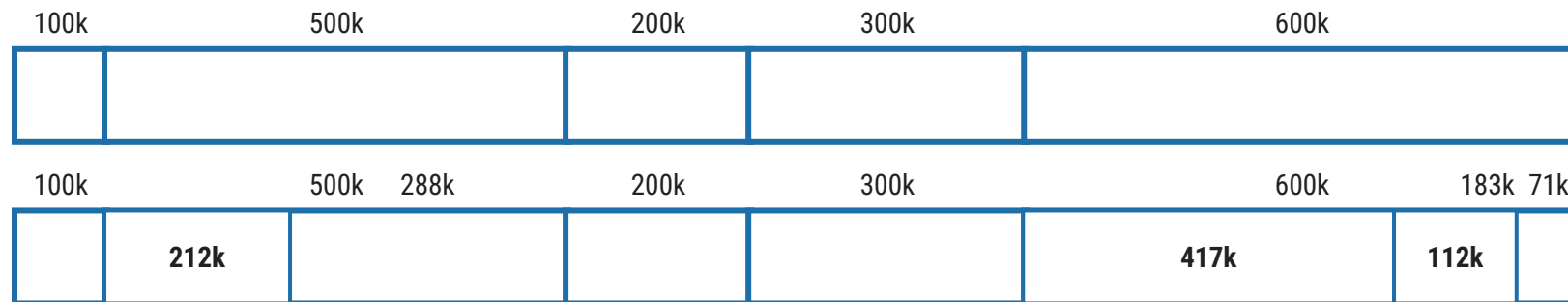
- ❑ Search **starts from the starting location** of the memory.
- ❑ **First available hole that is large enough to hold the process is selected** for allocation.
- ❑ The **hole is then broken up into two pieces**, one **for process** and another **for unused memory**.
- ❑ Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- ❑ Here process of size **426k will not get any partition** for allocation.
- ❑ **Fastest algorithm** because it searches as little as possible.
- ❑ **Memory loss is higher**, as very large hole may be selected for small process.

Next fit

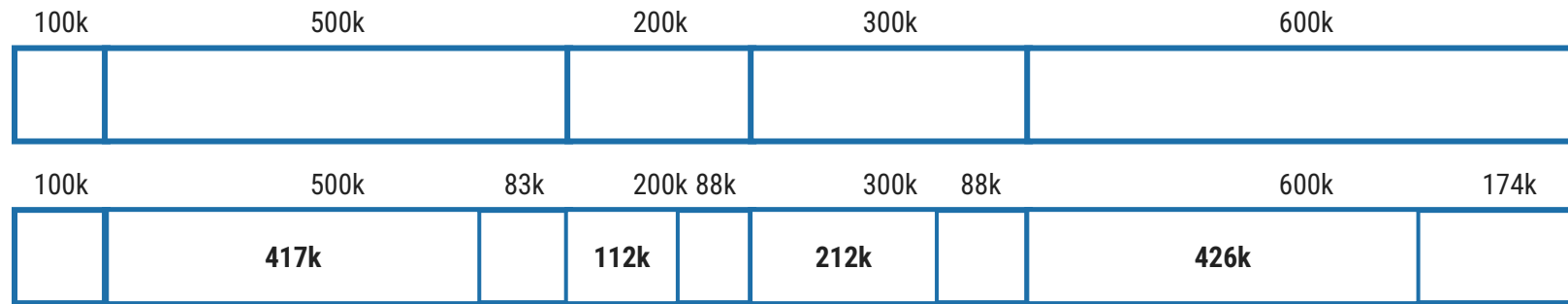
- It works in the same way as first fit, except that **it keeps the track of where it is whenever it finds a suitable hole.**
- The **next time when it is called to find a hole, it starts searching the list from the place where it left off last time.**
- Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- Here process of size **426k will not get any partition** for allocation.
- **Search time is smaller.**
- Memory manager must have to **keep track of last allotted hole to process.**
- It **gives slightly worse performance** than first fit.

Best fit

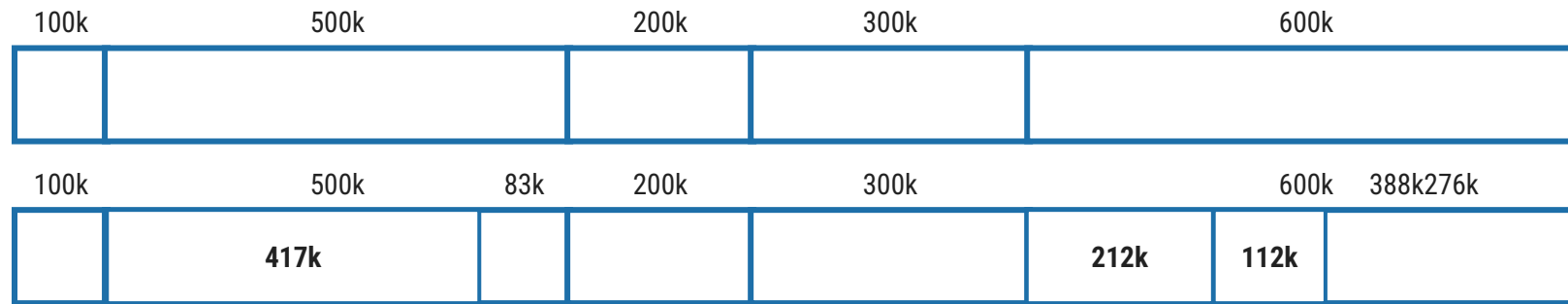
- ❑ Entire memory is searched here.
- ❑ The **smallest hole, which is large enough to hold the process, is selected** for allocation.
- ❑ Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- ❑ **Search time is high**, as it searches entire memory every time.
- ❑ **Memory loss is less.**

Worst fit

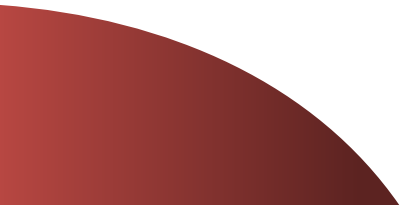
- ❑ Entire memory is searched here also. The **largest hole, which is largest enough to hold the process, is selected** for allocation.
- ❑ Example: Processes of **212K, 417K, 112K** and **426K** arrives in order.



- ❑ Here process of size **426k will not get any partition** for allocation.
- ❑ **Search time is high**, as it searches entire memory every time.
- ❑ This algorithm can be **used only with dynamic partitioning**.



Virtual Memory: Basics of Virtual Memory

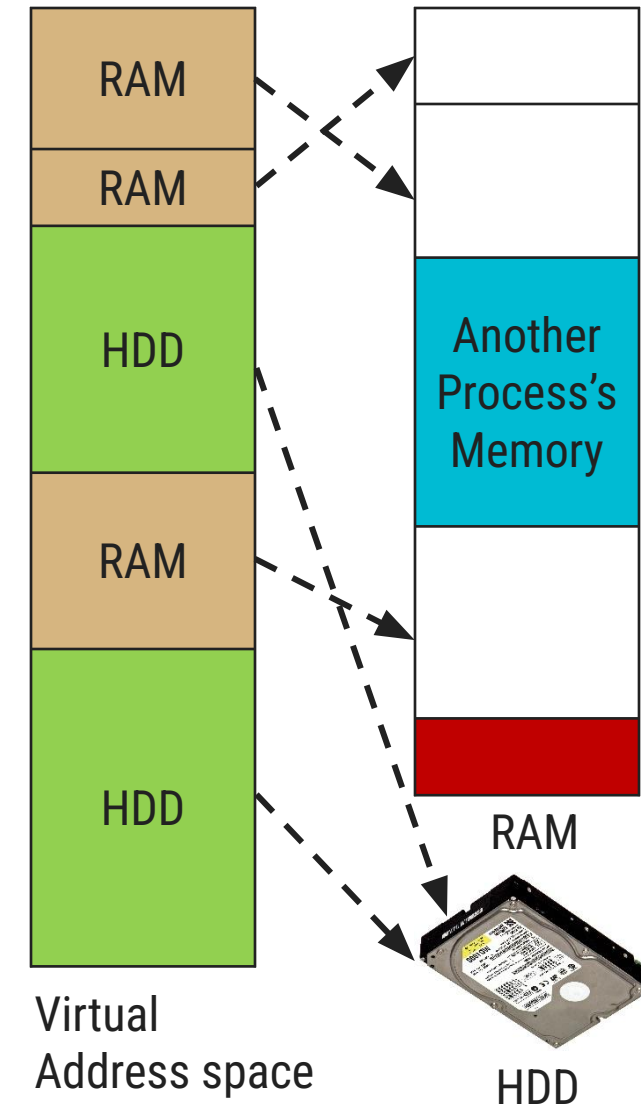


Virtual Memory

- ❑ Memory is **hardware that your computer uses to load the operating system and run programs.**
- ❑ Computer **consists of one or more RAM chips** that each have several memory modules.
- ❑ The amount of real **memory in a computer is limited** to the amount of RAM installed. Common memory sizes are **1GB, 2GB, and 4GB.**
- ❑ Because your computer has a finite amount of RAM, **it is possible to run out of memory when too many programs are running at one time.**
- ❑ This is where **virtual memory comes into the picture.**
- ❑ Virtual memory **increases the available memory of your computer by enlarging the "address space," or places in memory where data can be stored.**
- ❑ It does this by **using hard disk space** for additional memory allocation.
- ❑ However, since the **hard drive is much slower than the RAM, data stored in virtual memory must be mapped back to real memory in order to be used.**

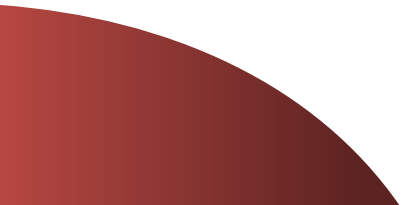
Virtual Memory

- ❑ Each program has its own address space, which is **broken up into pages**.
- ❑ Each **page is a contiguous range of addresses**.
- ❑ These **pages are mapped onto the physical memory** but, **to run the program, all pages are not required** to be present in the physical memory.
- ❑ The operating system **keeps those parts of the program currently in use in main memory**, and the rest on the disk.
- ❑ In a system using virtual memory, the **physical memory is divided into page frames** and the **virtual address space is divided into equally-sized partitions called pages**.
- ❑ Virtual memory **works fine in a multiprogramming system**, with bits and pieces of many programs in memory at once.





Paging



Paging

- Paging is a **storage mechanism used to retrieve processes from the secondary storage (Hard disk) into the main memory (RAM) in the form of pages.**

Virtual
Address
Space

40K – 44K	Page 11
36K – 40K	Page 10
32K – 36K	Page 9
28K – 32K	Page 8
24K – 28K	Page 7
20K – 24K	Page 6
16K – 20K	Page 5
12K – 16K	Page 4
8K – 12K	Page 3
4K – 8K	Page 2
0K – 4K	Page 1

Virtual page

- The main idea behind the paging is **to divide each process in the form of pages**. The **main memory will also be divided in the form of frames**.
- One page of the process is to be stored in one of the frames** of the memory.

Page frame

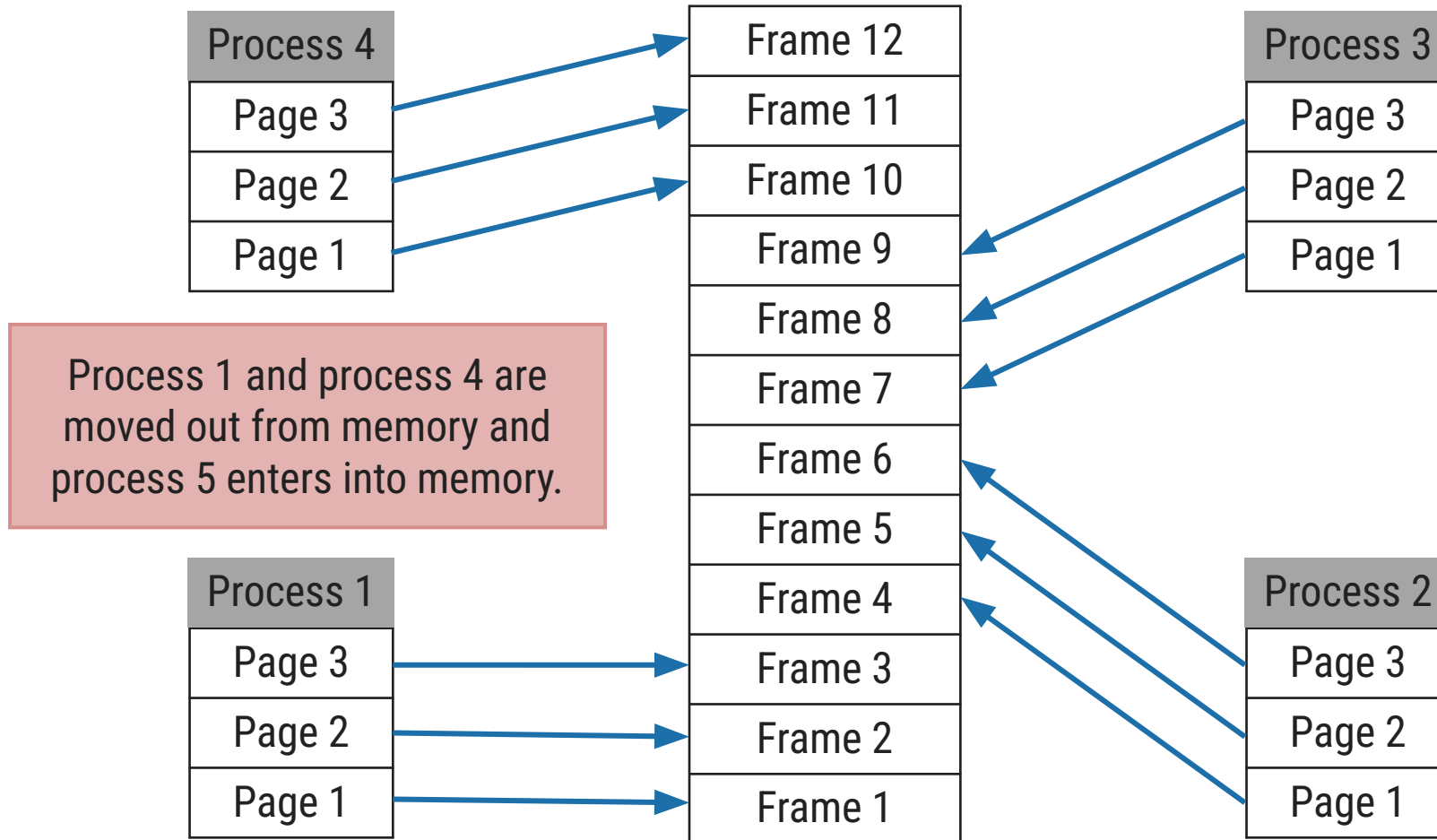
Frame 6	20K – 24K
Frame 5	16K – 20K
Frame 4	12K – 16K
Frame 3	8K – 12K
Frame 2	4K – 8K
Frame 1	0K – 4K

Physical
Memory
Address

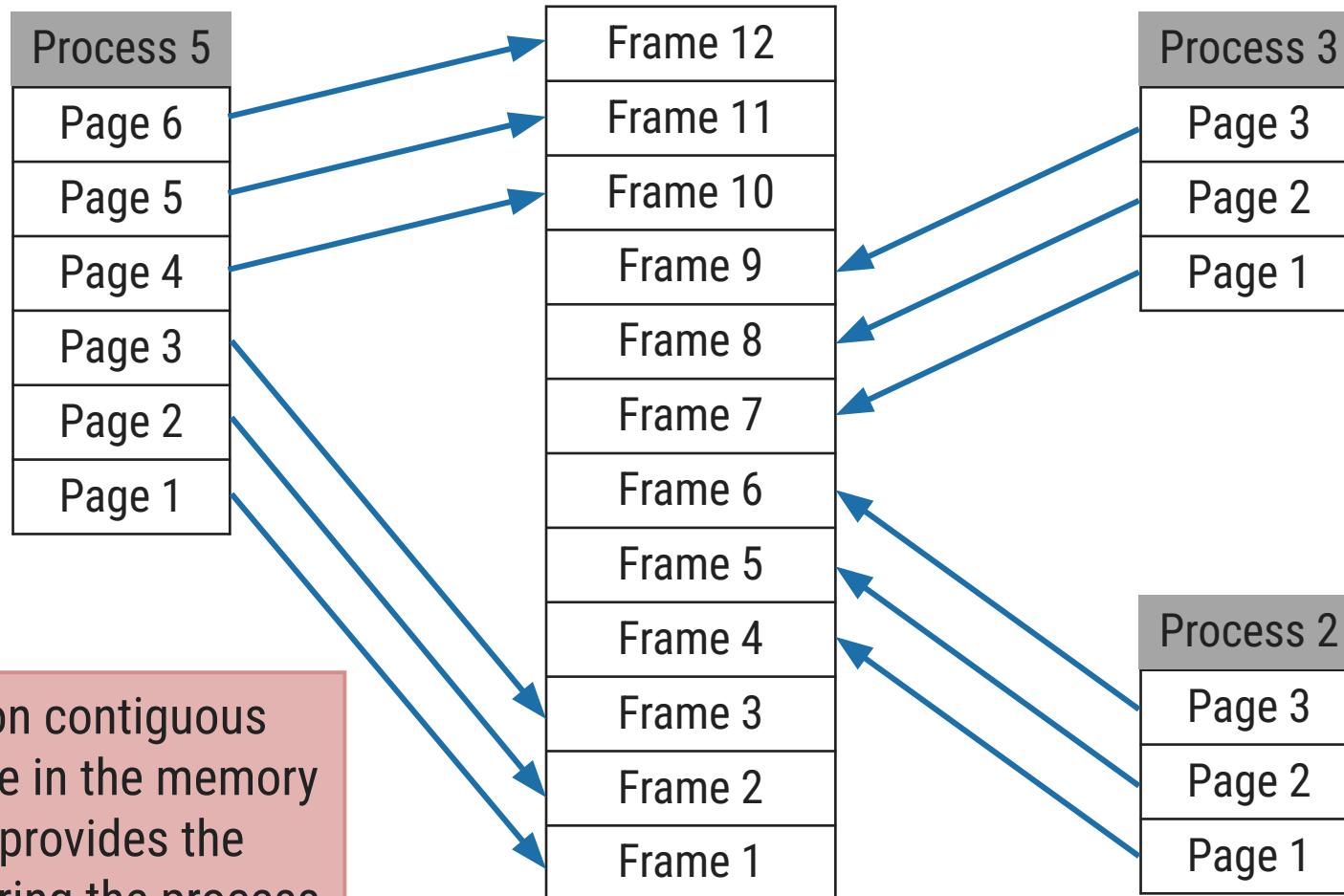
Paging

- ❑ The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes.
- ❑ Pages of the process are **brought into the main memory only when they are required** otherwise they reside in the secondary storage.
- ❑ The **sizes of each frame must be equal**. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.
- ❑ Different operating system defines different frame sizes.

Paging



Paging



We have 6 non contiguous frames available in the memory and paging provides the flexibility of storing the process at the different places.

Paging

- **Size of Virtual Address Space is greater than that of main memory**, so instead of loading entire address space in to memory to run the process, **MMU copies only required pages into main memory**.
- In order to **keep the track of pages and page frames**, OS **maintains a data structure called page table**.

Virtual
Address
Space

40K – 44K	
36K – 40K	
32K – 36K	5
28K – 32K	
24K – 28K	
20K – 24K	3
16K – 20K	4
12K – 16K	0
8K – 12K	
4K – 8K	1
0K – 4K	2

Virtual page

Page frame

5	20K – 24K
4	16K – 20K
3	12K – 16K
2	8K – 12K
1	4K – 8K
0	0K – 4K

Physical
Memory
Address

Logical Address vs Physical Address

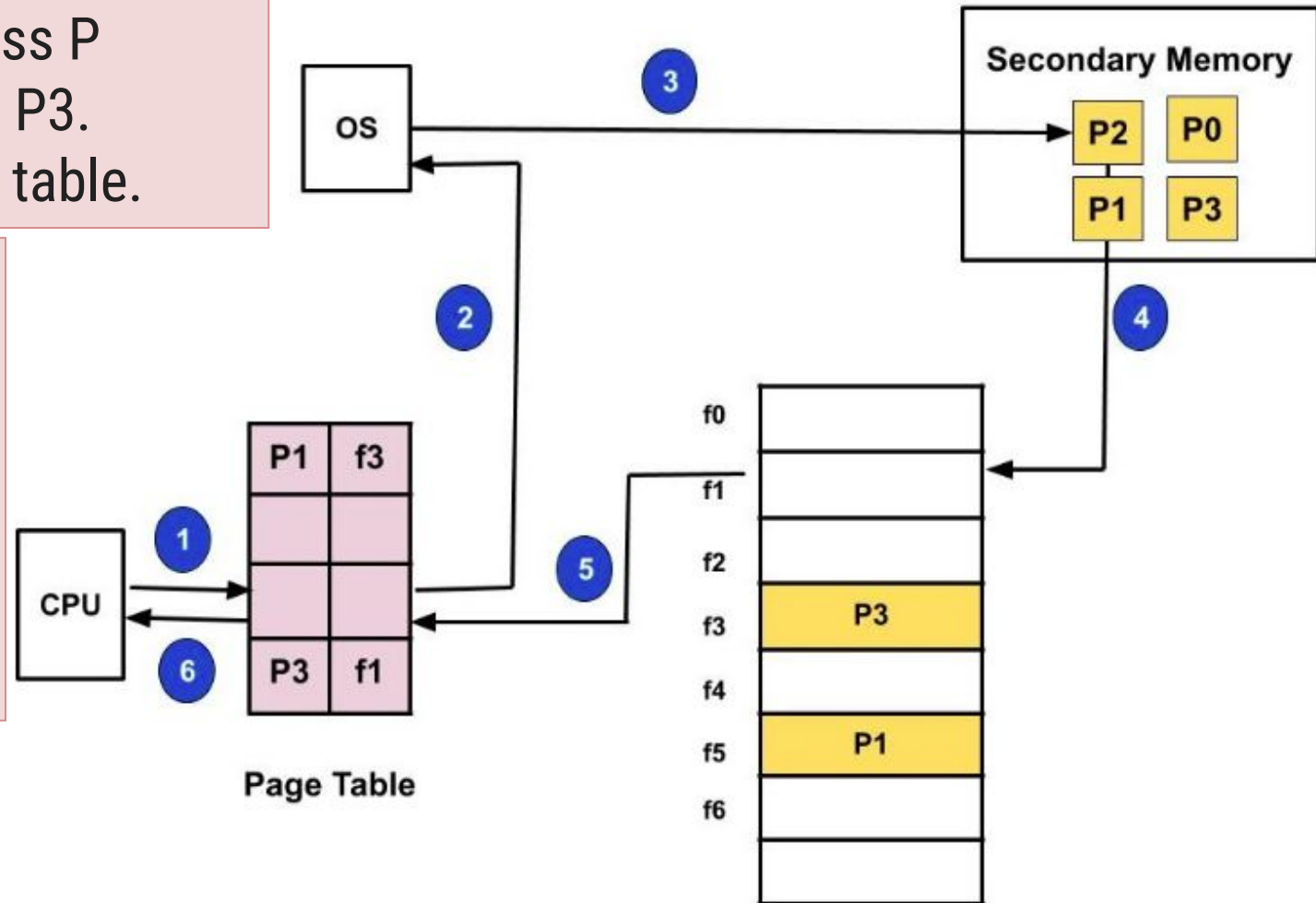
- ❑ **Logical Address is generated by CPU** while a program is running. The **logical address is virtual address as it does not exist physically** therefore it is also known as **Virtual Address**.
- ❑ **Physical Address identifies a physical location of required data in a memory**. The user never directly deals with the physical address but can access by its corresponding logical address.
- ❑ The **hardware device called Memory-Management Unit is used for mapping (converting) logical address to its corresponding physical address**.

Definitions (Demand paging)

□ **Demand paging:** Demand paging is a **technique used in virtual memory** systems where the **pages are brought in the main memory only when required or demanded by the CPU.**

- Suppose we have to execute a process P having four pages as P0, P1, P2, and P3.
- We have page P1 and P3 in the page table.

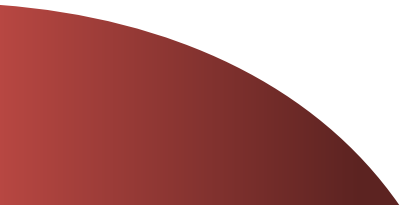
- Now, if the CPU wants to access page P2 of a process P
 - search the page in the page table
 - P2 is not available in page table so page fault
 - OS bring P2 from HDD to RAM
 - OS will update page table
 - Process P will be executed.





Page Replacement Algorithms

Section - 7



Page replacement algorithms

□ Following are different types of page replacement algorithms

1. Optimal Page Replacement Algorithm
2. FIFO Page Replacement Algorithm
3. LRU (Least Recently Used) Page Replacement Algorithm
4. NRU (Not Recently Used)

Optimal Page Replacement Algorithm

- The moment a page fault occurs, some set of pages will be in the memory.
- One of these pages will be referenced on the very next instruction.
- Other pages may not be referenced until 10, 100, or perhaps 1000 instructions later.
- Each **page can be labeled with the number of instructions that will be executed before that page is first referenced.**
- The optimal page algorithm simply says that the **page with the highest label should be removed.**
- The only problem with this algorithm is that it is **unrealizable.**
- At the time of the page fault, the **operating system has no way of knowing when each of the pages will be referenced next.**

Optimal Page Replacement Algorithm

□ Page Reference String:

- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 0, 2, 0, 1, 7, 0, 1
- Three frames

Page Request	7	0	1	2	0	3	0	4	2	3	0	3	2	0	2	0	1	7	0	1
Frame – 1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1
Frame – 2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
Frame – 3			1	1	1	3	3	3	3	3	3	3	3	3	3	3	3	7	7	7
Page Faults (9)	F	F	F	F		F		F			F						F	F		

FIFO Page Replacement Algorithm

- ❑ The first in first out page replacement algorithm is the simplest page replacement algorithm.
- ❑ The operating system **maintains a list of all pages currently in memory, with the most recently arrived page at the tail and least recent at the head.**
- ❑ On a page fault, the **page at head is removed and the new page is added to the tail.**
- ❑ When a page replacement is **required the oldest page in memory needs to be replaced.**
- ❑ The performance of the FIFO algorithm is **not always good because it may happen that the page which is the oldest is frequently referred by OS.**
- ❑ Hence removing the **oldest page may create page fault again.**

FIFO Page Replacement Algorithm

□ Page Reference String:

- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- Three frames

Page Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame – 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
Frame – 2		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
Frame – 3			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
Page Faults (15)	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

LRU (Least Recently Used) Page Replacement Algorithm

- ❑ A good approximation to the optimal algorithm is based on the observation that pages that have been heavily used in last few instructions will probably be heavily used again in next few instructions.
- ❑ When page fault occurs, **throw out the page that has been used for the longest time**. This strategy is called LRU (Least Recently Used) paging.
- ❑ To fully implement LRU, it is necessary to maintain a linked list of all pages in memory, with the **most recently used page at the front and the least recently used page at the rear**.
- ❑ The list must be updated on every memory reference.
- ❑ Finding a page in the list, deleting it, and then moving it to the front is a very time consuming operations.

LRU (Least Recently Used) Page Replacement Algorithm

□ Page Reference String:

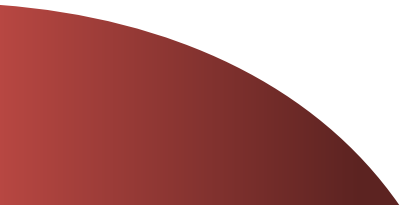
- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- Three frames

Page Request	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame – 1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
Frame – 2		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
Frame – 3			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
Page Faults (12)	F	F	F	F		F		F	F	F	F			F		F		F		



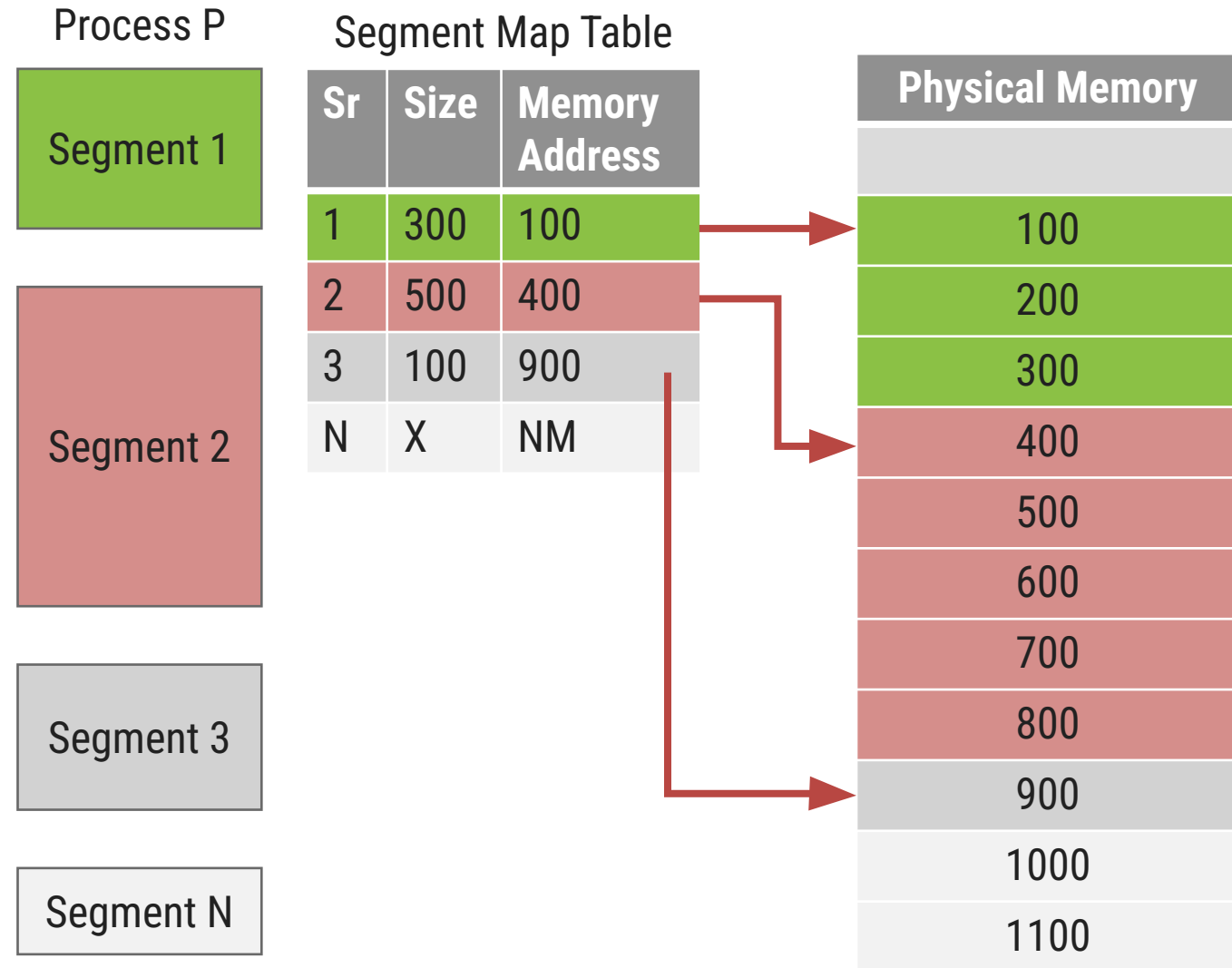
Segmentation

Section - 8



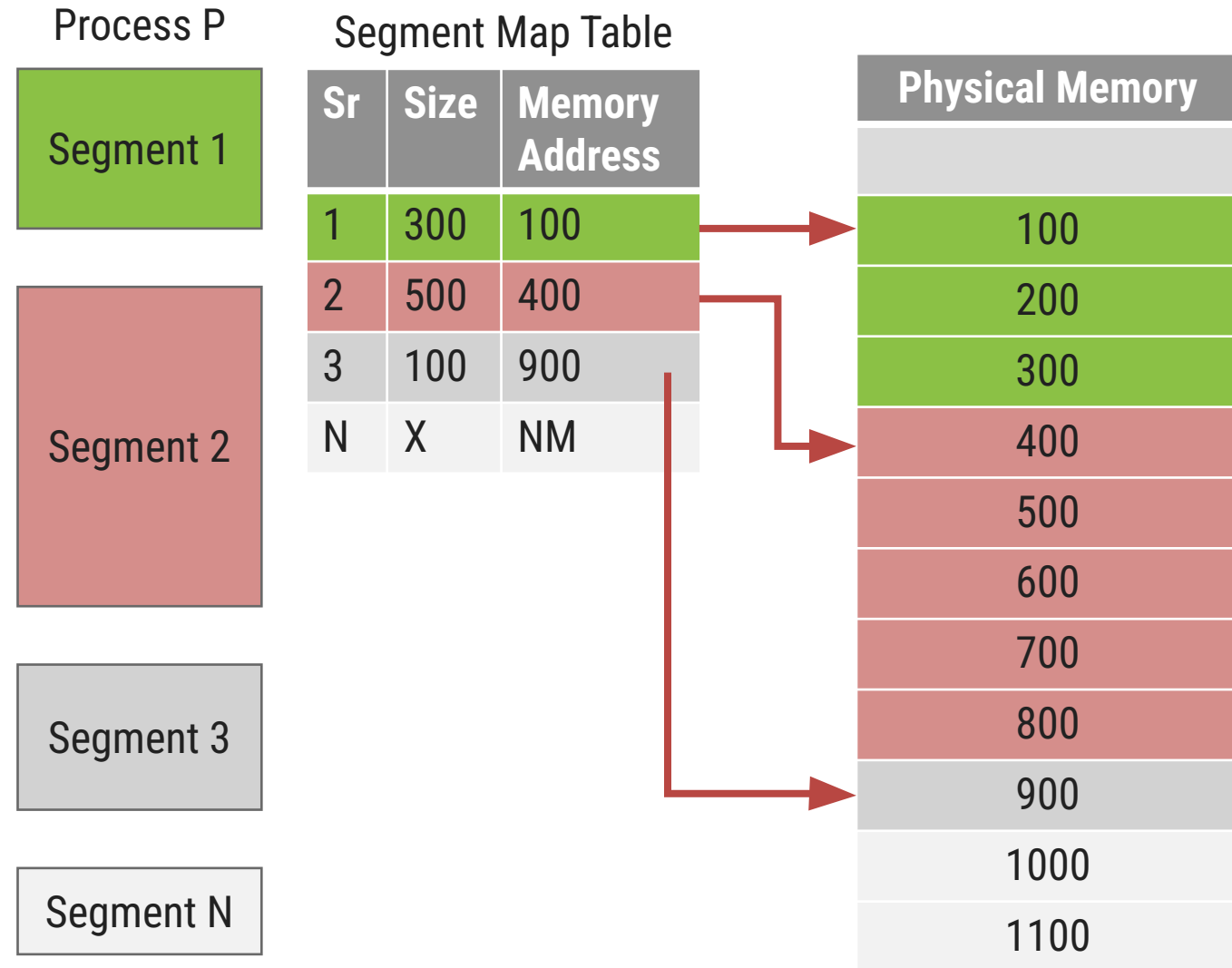
Segmentation

- ❑ Segmentation is a **memory management technique in which each job is divided into several segments of different sizes**, one for each module that contains pieces that perform related functions.
- ❑ **Each segment is actually a different logical address space** of the program.
- ❑ **When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory** though every segment is loaded into a contiguous block of available memory.
- ❑ Segmentation memory management works very similar to paging but here **segments are of variable-length where as in paging pages are of fixed size**.



Segmentation

- A program segment contains the program's main function, utility functions, data structures, and so on.
- The **operating system maintains a segment map table for every process.**
- **Segment map table contains list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory.**
- For each segment, the **table stores the starting address of the segment and the length of the segment.**
- A reference to a memory location includes a value that identifies a segment and an offset.



Paging VS Segmentation

Paging	Segmentation
Paging was invented to get large address space without having to buy more physical memory.	Segmentation was invented to allow programs and data to be broken up into logically independent address space and to add sharing and protection.
The programmer does not aware that paging is used.	The programmer is aware that segmentation is used.
Procedure and data cannot be distinguished and protected separately.	Procedure and data can be distinguished and protected separately.
Change in data or procedure requires compiling entire program.	Change in data or procedure requires compiling only affected segment not entire program.
Sharing of different procedures not available.	Sharing of different procedures available.



***Thank
You***

